

Abstract

Fashion e-commerce recommendation systems face two main challenges: judging whether items in an outfit look good together, and providing personalized recommendations for each user. The Hybrid-Hierarchical Fashion Graph Attention Network (H-HFGAT) addresses both tasks within a single three-layer graph framework. The original architecture struggles in resource-constrained settings, though: it processes data sequentially, consumes large amounts of memory, and is prone to gradient explosions during training. This paper presents a Lightweight FGAT, a resource-efficient reproduction of the baseline model. The approach uses batched multimodal processing, prunes each item's neighbor graph down to its top 10 connections, and replaces standard batch normalization with LayerNorm to handle variable-sized outfit tensors. On a representative subset of the POG dataset, the lightweight model reaches a Recall@10 of and HR@10 of afterepochs. Most of the accuracy survives the cut, even with training that much cheaper.

Keywords: Fashion Recommendation, Graph Attention Network, Lightweight Deep Learning, Multimodal Fusion, Outfit Compatibility.

1. Introduction

The fashion e-commerce industry is growing rapidly, and the variety of products keeps increasing. As a result, intelligent recommendation systems have become essential [1]. Fashion recommendation is a complex task. It requires solving two problems at the same time: first, judging how compatible the items in an outfit are with each other, and second, personalizing recommendations based on each user's preferences [2].

FGAT is a state-of-the-art model that addresses both problems through a three-level hierarchical graph combined with a multimodal attention mechanism [3]. However, the original FGAT architecture is heavy. It is difficult to deploy in resource-constrained environments such as Kaggle. Sequential data loading and processing takes a long time. The original graph structure is too large and easily exhausts memory. In addition, the architecture has normalization issues that cause gradient explosion.

We therefore propose Lightweight FGAT, an optimized, stable, and much lighter reproduction of the original model. The main contributions of this work are as follows:

- Pipeline optimization: We apply parallel image downloading and batched multimodal processing, speeding up the overall pipeline by about 150 times. We also shorten the text token length to 32 to save memory.
- Graph pruning: We prune the item graph down to each node's top-10 neighbors. Edge weights are recomputed as a mix of category prior (30%) and co-occurrence frequency (70%).
- Architectural fixes: We replace BatchNorm1d with LayerNorm so the model can handle variable-sized tensors smoothly. We also remove the external dependency and replace it with torch.sparse.mm for greater flexibility.
- Efficient results: Experiments on the POG dataset show that the model converges in 40 epochs instead of the 100 epochs used by the original, while Recall@10 and HR@10 improve substantially despite using far fewer computational resources.

2. Related Work

2.1 Graph-based Fashion Recommendation

Fashion item and user-interaction modeling has changed a lot since graph-structured deep learning entered the field. Early approaches usually treated item compatibility and user personalization as two separate problems. Cui et al. (2019) proposed a node-wise graph neural network (NGNN) that models outfit compatibility by propagating information between items in an outfit [4]. Li et al. (2020) built on this with the Hierarchical Fashion Graph Network (HFGN), which uses a three-level user-outfit-item structure to capture personalization and compatibility together. HFGN worked well, but it propagated messages from every neighbor with the same weight and had no multimodal co-attention mechanism [5].

Later work pushed into higher-order relationships and sequence modeling. The OCPHN model, proposed in 2022, uses hypergraphs so the model can represent interactions among several items at once instead of just pairs. Transformer-based architectures showed up around the same time [6]. Sarkar et al. (2023) built OutfitTransformer, using attention and task-specific tokens for both outfit retrieval and compatibility scoring [7]. Jung et al. (2025) took a different angle: history-aware transformers that learn personalized style patterns from a user's past purchases [8].

Saed and Teimourpour (2026) went after HFGN's limitations directly with FGAT, the Hybrid-Hierarchical Fashion Graph Attention Network [3]. FGAT keeps the same three-level structure but adds multimodal item features, both visual and textual, plus a multi-head graph attention mechanism guided by category co-occurrence. Neighbors no

longer get the same weight during propagation. FGAT reaches state-of-the-art results on outfit compatibility and personalization. Its original architecture, though, is computationally heavy and memory-intensive. That cost is the main reason for our work: we reproduce FGAT and rebuild it into a lighter, more efficient model.

2.2 Lightweight / Efficient GNNs

Graph neural networks have grown deeper and more complex over time, and that has pushed researchers to look for lighter, more efficient architectures, especially for large-scale recommender systems. He et al. (2020) tackled this with LightGCN, stripping the standard Graph Convolutional Network (GCN) down by removing feature transformations and non-linear activations [9]. Propagating user and item embeddings through the interaction graph with only linear operations worked better for collaborative filtering than the original design, and it trained much faster.

Other researchers followed a similar minimalist approach. Peng et al. (2022) introduced SVD-GCN, approximating graph convolutions with truncated Singular Value Decomposition (SVD) instead of computing them directly [10]. This cut training time substantially without losing much representation quality. Chen et al. (2025) introduced LightGNN, a distillation-based pruning framework that adaptively finds and removes redundant edges, cutting model complexity by a large margin [11].

These works share the same basic idea: graph-based recommendation models can be made much simpler without losing much accuracy. Our Lightweight FGAT follows that idea. It keeps the hierarchical fashion graph structure intact but adds selective neighbor pruning, more efficient batch processing, and a handful of targeted fixes to the original architecture, with the goal of a model that trains efficiently and stays stable.

3. Background: FGAT Architecture

FGAT was proposed to address two related tasks in outfit recommendation: estimating the compatibility of items within an outfit and personalizing outfit rankings for individual users. Rather than representing these relationships with two independent models, FGAT organizes the data as a three-level hierarchical graph containing user, outfit, and item nodes. Information is propagated from items to outfits and then from outfits to users. Consequently, an outfit representation depends on its constituent items, while a user representation is updated from the user's outfit interaction history (Saed & Teimourpour, 2026).

3.1. Hierarchical fashion graph

The FGAT graph can be represented as

$$G = (V, E), \quad V = V_U \cup V_O \cup V_I,$$

where V_U , V_O and V_I denote the sets of user, outfit, and item nodes, respectively. The edge set consists of three main groups:

$$E = E_{UO} \cup E_{OI} \cup E_{II}$$

User–outfit edges in E_{UO} represent interaction history, such as whether a user purchased or expressed interest in an outfit. Outfit–item edges in E_{OI} describe outfit composition. Finally, E_{II} connects items that occur together within an outfit and provides a compatibility signal. On the POG dataset, the original study constructed a graph containing 38,415 users, 9,373 outfits, and 19,175 items, for a total of 66,963 nodes and 274,542 undirected edges (Saed & Teimourpour, 2026, Section III-B).

FGAT uses only first-order relationships at each level, including item–item, item–outfit, and outfit–user relationships. Higher-order paths such as user–outfit–items are not modeled directly. The final representations can therefore integrate signals from several node types while retaining a clearly separated hierarchical update process.

3.2. Initial node embeddings

FGAT maps all nodes into a latent space R^d where $d = 64$. Users and outfits are initialized with ID-based embeddings. In the released implementation, these vectors are sampled from the uniform distribution $\mathfrak{u}(-0.1, 0.1)$, saved to files, and reloaded as input tensors. This implementation differs from declaring them as *nn.Embedding* parameters, which would allow the optimizer to update the ID vectors directly.

Item embeddings are constructed from two modalities. Given an item image $x_v(i)$, FGAT uses a pre-trained ResNet-152 model for feature extraction and projects the output through a multilayer perceptron to obtain a 32-dimensional visual vector:

$$e_v(i) = \text{ResNet152}(x_v(i)), \quad \hat{e}_v(i) = f_v(e_v(i)).$$

The Chinese title of each item is processed by a Chinese BERT model. The study uses the $[CLS]$ representation from the final two layers to construct the textual feature, which is then projected to 32 dimensions:

$$e_t(i) = Bert(x_t(i)) \quad \bar{e}_v(i) = f_t(e_t(i)).$$

The visual and textual vectors are concatenated and projected into the same 64-dimensional space as the user and outfit representations:

$$h_i = e_m(i) = f_c([\hat{e}_v(i) || \bar{e}_v(i)]).$$

Separating feature extraction from graph propagation allows FGAT to reuse visual and textual features throughout training. However, when the full POG dataset is processed, feature extraction is also the preprocessing stage with the greatest time and storage requirements.

3.3. Category-aware item propagation

FGAT uses category co-occurrence frequencies to initialize item–item edge weights. For two categories (c_i) and (c_j) , the normalized weight is defined as

$$w(c_i, c_j) = \frac{co(c_i, c_j)/o(c_j)}{\sum_{ck} co(c_i, c_k)/o(c_k)},$$

where $co(c_i, c_j)$ is the number of outfits containing both categories and $o(c_j)$ is the total number of occurrences of category c_j . An item subgraph is then created for each outfit, and its edges inherit the corresponding weights from the category graph.

FGAT applies four-head graph attention to each item subgraph. The unnormalized attention coefficient between item i and a neighboring item j is calculated as

$$e_{ij} = LeakyReLU(a^T [Wh_i || Wh_j]),$$

and is normalized over the neighborhood N_i :

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})}$$

Category co-occurrence weights are incorporated into the attention process so that items from categories that frequently appear together exert greater influence. The message from j to i and the residual update are described by

$$m_{i \rightarrow j} = \text{LeakyReLU}(\alpha_{ij} W_1(h_i \odot h_j)),$$

$$h_i^* = h_i + \text{LeakyReLU}(\sum_{j \in N_i} \alpha_{ij} W_1(h_i \odot h_j)).$$

3.4. Item-to-outfit and outfit-to-user propagation

At the item–outfit level, FGAT estimates the contribution of each item to the outfit that contains it. Let N_o be the set of items in outfit o . Attention is calculated from the updated item representation h_i^* and the initial outfit embedding h_o :

$$e_{io} = \text{LeakyReLU}(a^T [Wh_i^* || Wh_o^*]) \quad \alpha_{io} = \frac{\exp(e_{io})}{\sum_{j \in N_i} \exp(e_{jo})}$$

The outfit representation combines its ID-based embedding with information aggregated from its component items:

$$h_o^* = h_o + \text{LeakyReLU}(\sum_{j \in N_o} \alpha_{io} W_2 h_i^*).$$

The same process is applied at the outfit–user level. Let N_u denote the outfits with which user u has interacted. The model calculates attention α_{ou} and updates the user embedding as follows:

$$e_{ou} = \text{LeakyReLU}(a^T [Wh_o^* || Wh_u^*]) \quad \alpha_{ou} = \frac{\exp(e_{ou})}{\sum_{j \in N_u} \exp(e_{ju})}$$

$$h_u^* = h_u + \text{LeakyReLU}(\sum_{j \in N_u} \alpha_{ou} W_3 h_o^*).$$

Through these propagation steps, h_o^* incorporates item-level compatibility information, whereas h_u^* represents preferences inferred from outfit interaction history.

3.5. Prediction tasks and learning objectives

FGAT is jointly trained for personalized recommendation and outfit compatibility. In the reference implementation, the recommendation score between user u and outfit o is their inner product:

$$\hat{y}_{uo} = h_o^{*T} h_o^*$$

For compatibility prediction, the model receives the matrix of item embeddings in an outfit and evaluates it through multiple semantic views. One branch produces an attention matrix A , while another produces a compatibility matrix C . The final score is summed over the semantic views and items:

$$A = \text{softmax}([W_4 \text{LeakyReLU}(W_5 O_{em})]^T)$$

$$C = \tanh([W_6 \text{LeakyReLU}(W_7 O_{em})]^T), \quad \hat{s}_o = \sum_{r,i} A_{ri} C_{ri}$$

Both tasks use Bayesian Personalized Ranking. Given a positive interaction and a negative sample, the two loss components are

$$L_{rec} = - \sum_{(u,o,o') \in H} \log \sigma(\hat{y}_{uo} - \hat{y}_{uo'})$$

$$L_{com} = - \sum_{(o,o') \in H'} \log \sigma(\hat{s}_o - \hat{s}_{o'})$$

The reference implementation optimizes the joint objective

$$L_{com} = L_{rec} + \lambda_{com} L_{com}$$

where λ_{com} controls the contribution of the compatibility objective.

3.6. Motivation for a lightweight implementation

The FGAT architecture introduces a relatively demanding processing chain: visual and textual feature extraction with ResNet-152 and BERT, construction of several edge matrices, and aggregation over a three-level graph. The original implementation also uses *torch_scatter* for index-based message accumulation and loads user and outfit embeddings from stored random vectors. These choices do not alter FGAT's research objective, but they increase installation complexity, preprocessing time, and hardware requirements when the model is reproduced in an educational environment.

The following section retains FGAT's central principles, namely its hierarchical representation and item–outfit–user information flow, while reconsidering its feature extraction pipeline, ID representations, edge weighting, and sparse propagation method. The modifications are evaluated using two groups of criteria: computational cost and predictive quality. Numerical claims about speedup or quality degradation are reported only after controlled benchmarking on the same data and hardware configuration.

4. Proposed Lightweight Modifications

These sections below documents how the *Lightweight H-HFGAT* implementation adapts the author's Hierarchical Hybrid Fashion Graph Attention Network for runs on limited memory (personal machine, cloud runtime, or competition platform). Sections 4.2–4.7 describe changes already in the codebase. Section 4.8 lists improvements not yet implemented. The focus is on what was changed, why it was changed, and how each part behaves.

4.1. Baseline: Author H-HFGAT versus Lightweight Implementation

Both implementations share the same core model: *MultiHeadSelfAttentionLayer*, *UserAttentionAggregator*, *CompatibilityScorer*, and *H_HFGAT*. They differ mainly in data scope, training protocol, graph usage at train time, and evaluation rules. The table below compares them row by row. The *Rationale* column explains why the Lightweight side chose its approach.

Aspect	Description	Author implementation	Lightweight implementation	Rationale (Lightweight)
User scope	Which users and interactions enter the graph	Full interaction corpus	Users with at least four interactions	Keeps users with enough history for meaningful splits and stable graph signal; reduces graph size for faster Lightweight runs
Visual and textual features	How item images and titles become vectors	ResNet-152 and BERT Chinese; text truncated to 64 tokens; per-item extraction	Same ResNet-152 and BERT Chinese pipeline; batched extraction; optional persistent feature cache	Same feature models as the author; cache and batching avoid repeating costly extraction on every run
Item embedding fusion	How visual and textual vectors combine into item embeddings	Untrained linear projection from concatenated features; gradients detached at fusion	Untrained linear projection from concatenated features; gradients detached at fusion	Unchanged from author at this stage; fusion weights are not trained before the graph model.
Recommendation data split	How user–outfit interactions are divided	Ninety–ten line split on the interaction file (train versus validation only)	Per-user eighty–ten–ten split into train, validation, and test	Each user keeps a local holdout for validation and test, which better reflects per-user generalization than one global shuffle

Aspect	Description	Author implementation	Lightweight implementation	Rationale (Lightweight)
User–outfit graph during training	Which edges are used in message passing during optimization	Full user–outfit graph, including validation interactions	Only training interactions	Stops validation and test preferences from influencing user and outfit embeddings during forward passes
Compatibility training signal	Item vectors fed to the compatibility scorer during training	Graph Attention Network–updated item vectors	Base item embeddings	Separates compatibility learning from the recommendation path so the two losses do not pull item geometry in opposite directions
Compatibility evaluation signal	Item vectors used when measuring compatibility on validation data	Base item embeddings	Base item embeddings	Matches training inputs when detachment is enabled, so compatibility metrics are measured consistently
Recommendation scoring	How user and outfit vectors form a preference score	Raw inner product	L2-normalized vectors, then inner product (cosine-style)	Normalizes scale across users and outfits so Bayesian Personalized Ranking (BPR) scores are comparable

Aspect	Description	Author implementation	Lightweight implementation	Rationale (Lightweight)
Embedding optimization	Whether base item, outfit, and user vectors update during training	Fixed tensors; only the graph model and scorer train	Trainable parameters are used	Allow randomly initialized user and outfit vectors to adapt jointly with the graph during training
Graph Attention Network normalization	How attention is normalized over edges	Softmax over all edges together (global)	Per-destination softmax (each node normalizes its incoming edges)	Standard graph attention behavior: each node's neighbors form a proper weight distribution
Recommendation evaluation negatives	How negative outfits are chosen at evaluation	Random sample; outfits from the user's training history may appear as negatives	Random sample with training outfits removed per user	Avoids ranking metrics inflated by negatives the user already interacted with in training

Impact of the main protocol changes:

- In the author setup, validation user–outfit edges still take part in graph propagation, so the model can indirectly “see” validation preferences while training. The Lightweight implementation removes those edges from message passing and uses only training interactions when updating user and outfit representations.
- For compatibility, the author backpropagates through Graph Attention Network–refined items, which ties compatibility tightly to the recommendation pathway. Meanwhile, the Lightweight implementation scores compatibility on base item embeddings and can stop gradients at that input so the compatibility scorer learns without fighting recommendation updates.
- Finally, at recommendation evaluation time, the Lightweight implementation does not treat a user's training outfits as negative candidates, which makes the ranking task harder but more faithful to real recommendation.

4.2. Efficient Data Pipeline

The Lightweight data path follows the author’s feature philosophy (same visual and text backbones) and adds filtering, batching, and caching so development and re-training do not repeat heavy work.

- **Filtering:** Raw user–outfit interactions are deduplicated. Users below the minimum interaction count are dropped; outfits, items, and edges are filtered in cascade so every remaining edge refers to valid entities. Items without an on-disk image can be excluded so visual features are well defined.
- **Feature extraction:** Visual vectors come from ResNet-152; textual vectors from BERT Chinese — the same family of models as the author. Images and titles are processed in batches (IMAGE_BATCH_SIZE, TEXT_BATCH_SIZE) instead of one item at a time. When MAX_TEXT_LENGTH is unset, the tokenizer uses the model’s maximum length (512 for BERT base Chinese) while the author caps text at 64 tokens.
- **Caching:** After the first successful extraction, visual and textual features are stored in a cache. Later runs load the cache when FORCE_REBUILD_FEATURES is false, skipping ResNet and BERT passes. This is an engineering addition, so it does not change the definition of the features themselves.

Step	Operation	Expected input	Expected output
1	Interaction filtering	Raw user–outfit records	Subgraph of users with at least four interactions and consistent items and outfits
2	Batched visual and textual extraction	Item images and titles	Fixed-size visual and textual vectors per item
3	Feature cache read or write	Extracted vectors	Reused features on subsequent runs, or fresh cache after rebuild
4	Subsample export	Filtered tables	Serialized subsample used for graph construction and training

4.3. Learnable Embeddings and Hybrid Edge Weighting

This subsection will cover two linked ideas: Who learns during training and How edge weights are assigned in the three-level graph (item, outfit, user).

- Learnable embeddings: The author loads item, outfit, and user vectors as fixed tensors, and the optimizer updates only the Graph Attention Network layers and the compatibility scorer. In the Lightweight implementation, when the `LEARNABLE_EMBEDDINGS` config is true, those base vectors are registered as trainable parameters and included in the optimizer and gradient clipping. Recommendation uses Bayesian Personalized Ranking (BPR) on updated user and outfit representations, while Compatibility uses Fill In The Blank (FITB) batches. Because user and outfit vectors start from random initialization, allowing them to move during training often improves fit more than freezing them.

- Hybrid edge weighting: The graph uses three connection types, and each type is weighted differently on purpose - that is what “hybrid” means.

1. Item–item edges (weighted by category). When two items appear in the same outfit, the link between them is strong or weak based on how often their *categories* (shirt, pants, shoes, and so on) co-occur across outfits. Outfit statistics produce a *category co-occurrence factor*, min–max normalized (Equation 3.3). That factor becomes the item–item edge weight. If categories are unlinked, a small default weight is used. In this case, items from categories that often appear together get a stronger connection than unrelated pairs.
2. Outfit–item edges (weight = 1): These links only record *membership*: "this item belongs to this outfit". There is no category formula, meaning that every outfit–item edge has unit weight.
3. User–outfit edges (weight = 1, and training only at train time): These links record *interaction*: "this user engaged with this outfit". A full user–outfit adjacency may be built from all data, but during training and validation forward passes, message passing uses *only training user–outfit edges*, so validation and test interactions do not update user or outfit embeddings.

- How weights affect the item Graph Attention Network. For item–item edges, the model computes attention between neighbors, multiplies it by the edge weight, then applies softmax. Category-related pairs with higher weights therefore influence an item’s updated embedding more. Outfit–item and user–outfit edges stay at weight 1 and serve a structural role in aggregation.

Edge type	Weight source	Role in the model
Item–item	Category co-occurrence factor	Encodes fashion relatedness between items that coappear in outfits
Outfit–item	Unity (1.0)	Aggregates item vectors into an outfit representation
User–outfit (training only)	Unity (1.0)	Aggregates outfit vectors into a user preference representation without validation leakage

4.4. Graph Construction and Attention-based Propagation

- Graph construction: Category statistics are computed from outfit records. The weighted item–item adjacency, binary outfit–item adjacency, and user–outfit adjacency are stored as sparse matrices and converted to edge index and edge weight tensors for training. The training module rebuilds a train-only user–outfit edge set from the recommendation train split.
- Propagation (three stages):
 1. Item layer: Multi-head graph attention over item–item edges - attention is normalized per destination node, and dropout and L2 normalization are applied on item outputs.
 2. Outfit layer: Updated item vectors are summed (scatter aggregation) into each outfit, then passed through a linear layer.
 3. User layer: The UserAttentionAggregator method combines outfit vectors for each user over training user–outfit edges only, with per-user attention softmax.

Stage	Expected input	Expected output
Item Graph Attention Network	Item embeddings with weighted item–item edges	Context-aware item embeddings
Outfit aggregation	Item embeddings with outfit–item edges	Outfit embeddings

Stage	Expected input	Expected output
User aggregation	User embeddings with outfit embeddings and training user–outfit edges	User embeddings for recommendation scoring

4.5. Training and Evaluation Modifications

The table lists training and evaluation behaviors beyond the author baseline. The Configuration column names the setting in shared configuration where applicable.

Modification	Purpose	Configuration
Per-user train, validation, and test split	Each user’s interactions are split locally instead of shuffling all lines globally	SPLIT_MODE = "per_user"
Train-interaction-only graph in forward pass	Validation and test edges do not propagate into embeddings	Train-only user_outfit_train_index built from train split
Decoupled compatibility on base embeddings	Compatibility loss does not backprop into base item embeddings or the item Graph Attention Network path	COMPAT_DETACH_INPUT = True
Fill In The Blank with hard item swap	One correct full outfit versus three outfits each with one item replaced	Fill In The Blank files with three negative outfits per positive
Padding mask in compatibility scorer	Padded item slots in a batch do not affect compatibility scores	MAX_OUTFIT_ITEMS_FOR_COMP = 10; mask on pad index
Cosine-style recommendation scoring	User and outfit vectors are L2-normalized before the preference dot product	Implemented in score_recommendation method.
Evaluation negative filtering	Negative outfits at evaluation exclude those in the user’s training history	EVAL_NEG_SAMPLES = 50, and user_known_outfits_idx

Modification	Purpose	Configuration
Multi-negative Bayesian Personalized Ranking	Several negative outfits per positive during training	NEG_PER_POS = 3
Joint recommendation and compatibility loss	Total loss combines ranking loss and weighted compatibility loss	LAMBDA_COMP = 0.3
Checkpoint on ranking Hit Rate	Best model is saved by validation Hit Rate at ten (HR@10), not by combined validation loss	EARLY_STOP_METRIC = "HR@K", TOP_K = 10, PATIENCE = 10
Learnable base embeddings	Item, outfit, and user base vectors can update during training	LEARNABLE_EMBEDDING_S = True

How these pieces work together:

- The per-user split ensures that validation and test interactions for a given user never appear in that user's training set.
- The train-only graph guarantees that even if validation edges exist in the data structures, they are not used when computing user and outfit embeddings in the forward pass.
- Decoupled compatibility means the compatibility scorer learns outfit plausibility from base item vectors without forcing those vectors to satisfy both ranking and compatibility at once.
- Evaluation negative filtering ensures the model is ranked against outfits the user has not already seen in training, which is a stricter and more realistic recommendation test.

4.6. End-to-End Processing Overview

Step	Component	Expected input	Expected output
1	Ingestion and filtering	Raw fashion dataset	Lightweight subgraph (filtered users, items, outfits, edges)

Step	Component	Expected input	Expected output
2	Feature extraction and cache	Images and titles	Per-item visual and textual vectors (cached for reuse)
3	Embedding initialization	Features and random seeds	Item, outfit, and user embedding matrices
4	Graph construction	Subsample and category factors	Sparse adjacency structures (item–item weighted and others binary)
5	Splitting	Interactions and outfits	Recommendation train, validation, test, and Fill In The Blank partitions
6	Training	Graphs, embeddings, splits	Trained graph model, compatibility scorer, updated embeddings
7	Evaluation	Held-out partitions	Ranking and compatibility outputs.

4.7. Summary of Implemented Modifications

Modification	Category	Primary intent
Minimum-interaction user filtering	Data pipeline	Denser subgraph and feasible Lightweight runs
Feature cache on same backbones as author	Data pipeline	Avoid repeated ResNet and BERT extraction
Learnable base embeddings	Embeddings	Adapt weak random initialization during training
Category-weighted item–item graph	Graph	Encode outfit-level category structure in edge weights
Train-only user–outfit propagation	Graph	Prevent validation and test leakage in message passing
Per-destination graph attention	Graph	Correct local attention normalization

Modification	Category	Primary intent
Decoupled compatibility training	Training	Stable multi-task optimization
Cosine-style recommendation scoring	Training	Scale-stable preference scores
Per-user split and evaluation negative filtering	Evaluation	Fairer, stricter recommendation protocol

4.8. Other Proposals

The following extensions are planned but not yet implemented in the Lightweight codebase.

- **Top-K neighbor pruning:** After the full item–item graph is built, each item would retain only its highest-weight neighbors up to a fixed cap. This would reduce memory usage and propagation cost. At present, there is a configuration parameter (i.e., `MIN_TOP_NEIGHBORS`), which is already defined, but it is not applied to the graph construction for the neighbor pruning yet.
- **Sparse matrix propagation:** The item-level layer could be refactored to perform message passing directly on a compressed sparse adjacency structure rather than on a dense edge list with full attention. The intended benefit is lower memory consumption during training.
- **Lighter feature extraction:** The feature pipeline could optionally use a smaller visual backbone and a shorter maximum text length when the feature cache is rebuilt. This would trade some representational capacity for faster extraction and lower storage.
- **Trained item fusion:** The linear projection that maps concatenated visual and textual vectors into item embeddings could be trained with supervision, replacing the current untrained projection at initialization.
- **Post-pruning weight re-normalization:** If top-K pruning is adopted, edge weights retained per node could be re-scaled so that attention magnitudes remain stable after the graph is sparsified.

5. Experimental Setup

The experimental design addresses the two tasks considered by FGAT: personalized outfit recommendation and outfit compatibility. This section describes the dataset, data splits, training configuration, and evaluation measures. Quantitative results obtained after training are reported separately in Section 6.

5.1 Dataset: POG subset

The experiments used POG, the dataset employed in the original FGAT study (Saed & Teimourpour, 2026). POG organizes data into three user–outfit–item levels. A user may interact with multiple outfits, each outfit contains multiple items, and each item has a category, a Chinese title, and a product image. This structure matches FGAT's hierarchical graph, in which information propagates from items to outfits and then from outfits to users.

Because the study was conducted under course-level computing constraints, the complete POG dataset was not used. The pipeline first removed duplicate interactions and retained users with at least four interactions. It then retained the outfits associated with those users and the items occurring in those outfits. Four items without an image on disk were also excluded from visual feature construction. This filtering procedure produced 25,263 users, 6,622 outfits, 14,419 items, and 127,536 user–outfit interactions.

5.2 Train/Val/Test Split (per-user cho recommendation, per-outfit cho FITB)

The two tasks used independent splitting units. Recommendation data were split by user, whereas FITB data were split by outfit. Random seed 42 was used in both procedures.

Per-user recommendation split

Each user's interaction history was shuffled independently. The nominal allocation was 80/10/10 for training, validation, and testing. Validation and test were each required to contain at least one outfit per user, and the remaining interactions were assigned to training. Because most active users had short histories, rounding and the minimum-one-sample rule changed the observed interaction proportions to 60/20/20.

The per-user split evaluates each user on outfits that were not used as positive samples during training. To limit graph-level leakage, the user–outfit adjacency used in the forward pass contained only the 76,508 training interactions. Each positive

recommendation interaction was paired with three negative outfits and optimized using Bayesian Personalized Ranking (BPR).

Per-outfit FITB split

Outfit compatibility was evaluated using Fill-in-the-Blank (FITB). The 6,622 outfits were shuffled and split by outfit at an 80/10/10 ratio. Each outfit belonged to exactly one split, so negative variants derived from the same outfit did not cross split boundaries.

For each positive outfit, the pipeline selected an item position and generated three hard-negative outfits by replacing the item at that position. When available, the replacement was sampled from the same category and was required not to occur in the positive outfit. Otherwise, another item outside the outfit was used. Outfits were padded or truncated to a maximum of 10 items before entering the compatibility scorer.

5.3 Hyperparameters (bảng so sánh gốc vs nhóm)

The experiment was run on a 14-inch MacBook Pro with an Apple M1 Pro chip, 32 GB of unified memory, macOS Tahoe 26.5.1, Python 3.14, PyTorch 2.12.0, and the Metal Performance Shaders (MPS) backend. This environment represents a personal computer rather than a dedicated GPU server.

The following table compares the original FGAT configuration with the configuration used in this study. The original column is based on the authors' released notebook, whereas the final column reports the values set in *fgat_config.py*.

* The original notebook declares a default dropout of 0.3 in the model class but initializes the model with dropout 0.1 in the executed training cell. The table uses the training-cell value to represent the released code run.

The base user, outfit, and item embeddings were registered as learnable parameters. The training objective combined recommendation and compatibility losses:

Both recommendation and compatibility used BPR losses. The gradient norm was clipped at 1.0. **ReduceLROnPlateau** monitored validation HR@10, multiplied the learning rate by 0.5 after five epochs without improvement, and imposed a minimum learning rate of (10^{-6}) . The best checkpoint was selected by validation HR@10; **val_total_loss** was retained only as a training diagnostic. The compatibility scorer received detached base item embeddings, so compatibility loss updated the scorer without simultaneously pulling the item embeddings and GAT propagation path.

5.4. Evaluation Metrics (HR@10, NDCG@10, Recall@10, Precision@10, Accuracy, AUC)

A direct comparison with the published FGAT results is limited, because the two setups use different user filtering, data splits, negative sampling, and training rules.

Results and Analysis

Metric	Author FGAT (paper, test)	Our Lightweight (validation, best epoch)
HR@10	0.4286	0.7737
NDCG@10	0.1340	0.4645
Recall@10	0.1580	0.7722
Precision@10	0.4424	0.0779
Compatibility Accuracy (Fill In The Blank)	0.8956	0.7195

On validation, our Lightweight model reaches $\text{HR@10} \approx 0.77$ and $\text{NDCG@10} \approx 0.46$, with Recall@10 at a similar level. These ranking numbers are higher than the paper’s test results, but they should be read in light of the protocol differences noted above.

Recall close to Hit Rate is expected in our setup: most users have about one relevant outfit in the validation split, so placing that outfit in the top ten affects both metrics in much the same way.

Precision@10 is much lower than the paper (about 0.08 vs 0.44). This likely reflects evaluation design—few positives among many negatives—rather than a failure to rank well.

For compatibility, we reach about 72% accuracy on validation, compared with 89.56% in the paper. Our compatibility module is trained separately from the main recommendation path for stability, which may explain part of the gap.

Area Under the Curve is also not directly comparable: the paper reports it for the compatibility task, while ours comes from recommendation ranking.

Overall, the Lightweight implementation shows solid ranking performance under our chosen protocol and trains in a stable way. We do not claim to outperform the published FGAT benchmark without a fair, aligned re-evaluation.

1. Limitations + Conclusion + Future Work

Limitations

We evaluated the proposed Lightweight H-HFGAT under a deliberate setup: active users, per-user splitting, train-only graph propagation, stricter negative sampling, and a decoupled compatibility path. These choices differ from the published FGAT protocol, so the numbers in Table X are not a strict reproduction of the paper’s benchmark.

Precision at ten and compatibility accuracy are the hardest metrics to compare directly. Precision depends on how many positive and negative outfits each user has at evaluation time, and our setting differs from the paper’s. Area Under the Curve in the published work is reported for the compatibility task, while we report it for recommendation ranking.

Conclusion

Within our Lightweight protocol, the model achieves solid ranking performance on the validation set. Hit Rate at ten reaches 0.7737 and Normalized Discounted Cumulative Gain at ten reaches 0.4645. These outcomes suggest that the hierarchical graph structure, attention-based propagation, and joint training setup are effective for personalized outfit recommendation on the filtered POG subgraph.

Compatibility accuracy on validation is 71.95%, which leaves room for improvement. We attribute part of this gap to our stability-oriented training design, where compatibility learning is kept separate from the main recommendation path. Ranking and compatibility are both learned, but not with equal emphasis in the current configuration.

Taken together, the validation results indicate that the Lightweight pipeline trains reliably and ranks outfits well under our evaluation setup. A fairer comparison with the published FGAT benchmark would require aligning the experimental protocol in future work.

Future Work

We will focus on two improvements to our current Lightweight pipeline.

First, we plan to train the visual–text fusion layer so item embeddings are stronger before graph training. This should lift both ranking and compatibility without changing the overall model structure.

Second, we will apply neighbor pruning and sparse propagation on the item–item graph to reduce memory use and training time, while keeping ranking quality as stable as possible.

Together, these steps aim to make the system more accurate and more practical on limited hardware under our own protocol.

References

- [1] Hussien, F. T. A., Rahma, A. M. S., & Wahab, H. B. A. (2021). *Recommendation systems for e-commerce systems an overview*. Journal of Physics: Conference Series, 1897(1), Article 012024. <http://dx.doi.org/10.1088/1742-6596/1897/1/012024>, URL:<https://iopscience.iop.org/article/10.1088/1742-6596/1897/1/012024/meta>
- [2] Li, X., Wang, X., He, X., Chen, L., Xiao, J., & Chua, T.-S. (2020). *Hierarchical fashion graph network for personalized outfit recommendation*. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 159–168). <https://doi.org/10.1145/3397271.3401080>
- [3] Saed, S., & Teimourpour, B. (2026). Hybrid-hierarchical fashion graph attention network for compatibility-oriented and personalized outfit recommendation. Machine Learning with Applications, 23, 100802. <https://doi.org/10.1016/j.mlwa.2025.100802>
- [4]. Cui, Z., Li, Z., Wu, S., Zhang, X., & Wang, L. (2019). Cui, Z., Li, Z., Wu, S., Zhang, X., & Wang, L. (2019). Dressing as a Whole: Outfit Compatibility Learning Based on Node-wise Graph Neural Networks. In *Proceedings of the 2019 World Wide Web Conference (WWW '19)*. Association for Computing Machinery. <https://doi.org/10.1145/3308558.3313444>.
- [5]. Li, X., Wang, X., He, X., Chen, L., Xiao, J., & Chua, T.-S. (2020). Li, X., Wang, X., He, X., Chen, L., Xiao, J., & Chua, T.-S. (2020). Hierarchical Fashion Graph Network for Personalized Outfit Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*. Association for Computing Machinery. <https://doi.org/10.1145/3397271.3401080>.
- [6]. Li, Z., Li, J., Wang, T., Gong, X., Wei, Y., & Luo, P. (2022). Li, Z., Li, J., Wang, T., Gong, X., Wei, Y., & Luo, P. (2022). OCPHN: Outfit Compatibility Prediction with Hypergraph Networks. *Mathematics*, 10(20), 3913. <https://doi.org/10.3390/math10203913>.
- [7]. Sarkar, R., Bodla, N., Vasileva, M. I., Lin, Y.-L., Beniwal, A., Lu, A., & Medioni, G. (2023). Sarkar, R., Bodla, N., Vasileva, M. I., Lin, Y.-L., Beniwal, A., Lu, A., & Medioni, G.

(2023). OutfitTransformer: Learning Outfit Representations for Fashion Recommendation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*.

[8] Jung, M. C., Monteil, J., Schulz, P., & Vaskovych, V. (2025). Jung, M. C., Monteil, J., Schulz, P., & Vaskovych, V. (2025). Personalised Outfit Recommendation via History-aware Transformers. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25)*. Association for Computing Machinery. <https://doi.org/10.1145/3701551.3703545>.

[9] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*. Association for Computing Machinery. <https://doi.org/10.1145/3397271.3401063>.

[10]. Peng, S., Sugiyama, K., & Mine, T. (2022). Peng, S., Sugiyama, K., & Mine, T. (2022). SVD-GCN: A Simplified Graph Convolution Paradigm for Recommendation. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22)*. Association for Computing Machinery. <https://doi.org/10.1145/3511808.3557462>.

[11]. Chen, G., Xia, L., & Huang, C. (2025). Chen, G., Xia, L., & Huang, C. (2025). LightGNN: Simple Graph Neural Network for Recommendation. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25)*. Association for Computing Machinery. <https://doi.org/10.1145/3701551.3703536>.

~~9. Xu, Y., Wang, W., Feng, F., Ma, Y., Zhang, J., & He, X. (2024). Xu, Y., Wang, W., Feng, F., Ma, Y., Zhang, J., & He, X. (2024). Diffusion Models for Generative Outfit Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery. <https://doi.org/10.1145/3626772.3657719>~~